

Résolution de problèmes – TP Multi-objectifs

Janvier 2026

Pour la partie portant sur l'optimisation multi-objectifs, nous nous intéressons toujours au problème du flowshop de permutation (FSP), mais cette fois dans une variante à deux objectifs (mFSP). Le premier objectif (f_1) est la date de fin d'ordonnancement (*makespan*, C_{max}), tandis que le second objectif (f_2) est la somme des retards (*tardiness*, T_{sum}).

Dans les fichiers d'instances utilisés, certaines valeurs précédemment ignorées représentent en réalité les dates de fin souhaitées pour chaque job. L'objectif f_2 est de minimiser la somme des retards accumulés par l'ensemble des jobs. Notez qu'un job terminé en avance a un retard de 0 (et non une valeur négative).

```

1 20          % nombre de jobs
2 5           % nombre de machines
3 873654221   % seed (ignorer)
4 0           % job 0 (identifiant)
5 468         % *** date de fin souhaitee pour le job 0 ***
6 54 79 16 66 58 % duree du job 0 sur chaque machine : t01, t02, t03, t04, t05
7 1           % job 1
8 325         % *** date de fin job 1 ***
9 83 3 89 58 56 % t11, t12, t13, t14, t15
10 2          % job 2
11 923        % *** date de fin job 2 ***
12 15 11 49 31 20 % t21, t22, t23, t24, t25
13 3          % job 3
14 513        % *** date de fin job 3 ***
15 71 99 15 68 85 % t31, t32, t33, t34, t35
16 4          % job 4
17 1070       % *** date de fin job 4 ***
18 77 56 89 78 53 % t41, t42, t43, t44, t45
19 ...

```

A Partie 1A

Question 1

Mettre à jour votre fonction qui charge en mémoire les données d'une instance de problème pour intégrer les dates de fin souhaitées.

Question 2

Écrire une fonction `cout_T` qui retourne le coût d'une solution π , exprimé cette fois-ci comme la somme des retards. Pensez à vérifier la validité de votre fonction avec des petites instances exemples, créées à la main.

Question 3

Développer une fonction `eval_mo` qui calcule le coût d'une solution π passée en paramètre pour chacun des objectifs $f(\pi) = (f_1(\pi), f_2(\pi)) == (C_{max}(\pi), T_{sum}(\pi))$.

Question 4

Générer une solution aléatoire et l'évaluer. Comparer sa qualité avec la meilleure solution que vous avez identifiée pour le FSP mono-objectif, projetée dans l'espace objectif (f_1, f_2) .

B Partie 1B — Filtres

Un des défis majeurs de l'optimisation multi-objectifs consiste à filtrer efficacement les solutions dominées parmi un ensemble de points, afin de ne conserver que les solutions non-dominées (celles qui nous intéressent) :

- Dans une stratégie **off-line**, tous les points à filtrer sont connus simultanément. L'algorithme prend un ensemble de solutions en entrée et retourne le sous-ensemble de solutions non-dominées.
- Dans une stratégie **on-line**, l'algorithme maintient un ensemble de solutions appelé archive A (initialement vide). À chaque itération, une nouvelle solution arrive. Pour toute nouvelle solution π reçue en entrée, l'algorithme doit décider s'il l'ajoute à l'archive A et, le cas échéant, supprimer de A les solutions qu'elle domine.

Nous pourrions tester les algorithmes de filtrage sur des solutions du problème mFSP (pour le moment générées aléatoirement), puis vérifier visuellement leur bon fonctionnement à l'aide d'outils graphiques (tels que `gnuplot` ou `matplotlib`).

Question 5

Développer une fonction `filtrage_offline`. Pour chaque instance, générer 500 solutions aléatoires et filtrer les solutions dominées en utilisant votre algorithme hors ligne. Vérifier le bon fonctionnement de l'algorithme.

Question 6

Développer une fonction `filtrage_online`. Pour chaque instance, générer P solutions aléatoires et mettre à jour l'archive séquentiellement avec chacune de ces solutions (une à la fois). Analyser le nombre de comparaisons effectuées et le temps d'exécution de votre algorithme en fonction de P .

C Partie 2 — Algorithmes pour le mFSP

Pour résoudre le mFSP, nous allons considérer deux types d'approches : une approche scalaire et une approche Pareto.

Une approche **scalaire** recherche une solution Pareto approchée de manière itérative. Elle utilise une fonction d'agrégation pour transformer le problème multi-objectifs en un problème mono-objectif, puis ajuste progressivement les paramètres de cette fonction. Par exemple, avec une somme pondérée, différents poids sont définis pour explorer le front Pareto. Une heuristique mono-objectif peut être appliquée pour chaque vecteur de poids afin d'obtenir une solution. L'ensemble des solutions est ensuite filtré pour éliminer celles qui sont dominées. Concernant l'heuristique à utiliser, choisissez la méthode qui vous semble la plus pertinente par rapport à votre travail sur le FSP mono-objectif (quel voisinage ? quel climber ?).

Une approche **Pareto** maintient un ensemble de solutions et non une solution unique. En se basant sur la relation de dominance Pareto, cette approche vise à améliorer toutes les solutions simultanément plutôt que l'une après l'autre. Ce processus s'appuie souvent sur une archive (un filtre on-line). À chaque itération, une **recherche locale** Pareto génère de nouvelles solutions via un opérateur de voisinage, puis les ajoute à l'archive. Un filtre met ensuite à jour cette archive en ne conservant que les solutions non-dominées.

Question 7

Développer un algorithme **scalaire** pour le mFSP. Analyser son comportement et identifier clairement ses paramètres ainsi que ses différentes variantes.

Question 8

Développer un algorithme **Pareto** pour le mFSP. Analyser son comportement et identifier clairement ses paramètres ainsi que ses différentes variantes.

D Partie 3 — Analyse expérimentale

Pour évaluer et comparer les performances de vos algorithmes, nous allons procéder à l'analyse expérimentale suivante. Comme en optimisation mono-objectif, il faut d'abord définir un critère d'arrêt, typiquement un temps d'exécution ou un nombre maximum d'évaluations (par exemple, 1 million d'évaluations). Pour chaque instance, vous exécuterez vos algorithmes plusieurs fois (au moins 10 fois). Pour chaque instance et chaque algorithme, nous obtiendrons ainsi 10 approximations du front Pareto qu'il s'agit de comparer. Une première approche consiste à visualiser et comparer graphiquement les fronts obtenus. Une méthode plus rigoureuse repose sur les indicateurs de qualité vus en cours (comme l'hypervolume ou l'indicateur epsilon).

Question 9

Décrire précisément vos **conditions expérimentales** (variantes algorithmiques, paramètres, instances, critère d'arrêt, etc.) : vos analyses doivent être reproductibles !

Question 10

Exécuter chaque algorithme plusieurs fois sur chaque instance. Analyser **graphiquement** les fronts obtenus, puis commenter les figures et interpréter les résultats.

Question 11

Sélectionner un ou plusieurs **indicateurs** de qualité et mesurer leurs valeurs pour chaque exécution d'algorithme sur chaque instance. Présenter les résultats sous forme de figures et tableaux clairs et synthétiques. Commenter et interpréter les résultats.